



中国银联
China UnionPay

Python 实战大课堂 · 2026

Python 实战大课堂·2026

Python 数据处理与小模型应用实战

人工智能研发团队 吴枫

2026 年 9 月 24 日

课程目录: 三大核心模块

先看全貌, 再进实战

- 1 NumPy 与 Pandas: 业务表格与批量计算
- 2 小模型微调: 让 AI 听懂你的业务分类
- 3 ONNX 极简部署: 让小模型真正上线运行
- 4 全课回顾: 业务实战极简清单

三大阶段全景递进

- **第一章: NumPy & Pandas 表格实战**
清洗脏数据、分组汇总、替代 VLOOKUP 关联。
- **第二章: 小模型意图分类微调**
文本切词转数字、衡量差距、自适应纠偏训练。
- **第三章: ONNX 生产极简部署**
导出单文件模型, CPU 毫秒级无依赖极速推理。

全景架构：数据处理与小模型落地全流程

一页掌握端到端全景



通俗总结：先用 NumPy/Pandas 把杂乱的业务表格清洗规整；再让 **轻量小模型** 读懂客服意图；最后导出为 **ONNX**，用极小的服务器资源实现毫秒级快速响应！

MODULE 01

NumPy & Pandas

业务表格与批量数据处理

告别低效的原生 Python 循环与卡顿的手工 Excel，掌握面向千万行数据的全自动批处理技能。

【本章节核心解决的问题】

- **批量算账运算**：用 1 行代码代替多层嵌套 for 循环；
- **缺失与脏数据防御**：一键填充空白值、剔除退款冲正单；
- **分组报表聚合**：秒级跑出多维度的业务总额与平均流水；
- **多表跨表关联**：彻底替代传统 Excel 易错易卡的 VLOOKUP。

NumPy 是什么？主要能用来干什么？

第一章：核心定位与直观价值

【为什么数据处理首选 NumPy?】

- 普通 Python 列表的痛点：
 - 算一组数据必须写 for 循环一个个取出来算，代码冗长且运行很慢。
- NumPy 的核心能力：
 - 把一整列数字当成一个整体，整批计算。
- 初学者记住它的三大本领：
 - ① 批量算账：全列直接做加减乘除，一行代码搞定。
 - ② 快速统计：求和、均值、最大最小值瞬间得出。
 - ③ 条件筛选：像漏斗一样批量滤出需要的数据。

【循环 vs 批量算账对比】

业务需求：所有商品金额打 9 折，并统一加 5 元配送费。

原生写法：必须建立空列表，逐行 append 计算

NumPy 写法：直接 `prices * 0.9 + 5.0` 整批出结果！

```
import numpy as np

prices = np.array([100.0, 250.0, 80.0, 300.0])

# 1. 批量计算：所有金额打 9 折并加 5 元
final_prices = prices * 0.9 + 5.0
print("计算结果:", final_prices)
# 输出: [ 95.  230.   77.  275.]

# 2. 常用统计：均值与总额
print("平均额:", prices.mean(), "总额:", prices.sum())
```

NumPy 实用小技巧: 批量条件筛选与打标

第一章: 业务常见小技巧

【日常业务中最常用的两个用法】

● 技巧 1: 一秒筛选出正常数据

- 比如收集上来的数据有负数或 0 属于异常脏数据。
- 只要写 `amounts[amounts > 0]` 就能一次性挑出所有有效数值。

● 技巧 2: 类似 Excel 的 IF 批量打标

- 满足条件的标注为 A, 不满足的标注为 B。
- 使用 `np.where(条件, 满足时的值, 不满足时的值)`。

【代码实战演示】

```
import numpy as np

amounts = np.array([120.0, -10.0, 350.0, 0.0,
                    500.0])

# 1. 批量筛选: 过滤掉 <= 0 的异常记录
valid = amounts[amounts > 0]
print("有效金额:", valid)

# 2. 类似 Excel IF: 大于等于 200 算"大单", 否则"普单"
tags = np.where(amounts >= 200, "大单", "普单")
print("订单等级:", tags)
# 输出: ['普单' '普单' '大单' '普单' '大单']
```

Pandas 能解决什么问题?

第一章: 代码版超级 Excel

【初学者这样理解 Pandas】

- **核心概念:**
 - DataFrame: 一张有列名、有行号的规整二维表格。
 - Series: 单独抽出来的一整列。
- **为什么选择 Pandas 处理业务?**
 - 解决 Excel 行数过大卡顿、公式引错错位的问题。
 - 日常报表工作写成脚本, 一键自动化刷新。
- **核心实战三部曲:** 清洗过滤 → 分组汇总 → 跨表合并。

【DataFrame 的直观模样】

DataFrame 类似业务明细底表:

索引	用户 ID	所属部门	消费金额
0	U101	运营部	150.0
1	U102	研发部	320.0
2	U103	运营部	80.0
3	U104	市场部	400.0

```
import pandas as pd

df = pd.DataFrame({
    "用户 ID": ["U101", "U102", "U103", "U104"],
    "所属部门": ["运营部", "研发部", "运营部", "市场部"],
    "消费金额": [150.0, 320.0, 80.0, 400.0]
})

print(df)
```

实战流程: Pandas 业务表格处理四步法

第一章: 解决日常 90% 的数据整理需求



业务场景真实对应:

- **清洗筛选:** 去掉金额 ≤ 0 的退款单, 客户“所属地区”为空的自动填入“未知”;
- **分组统计:** 一键跑出“各个部门的总花费是多少”、“各渠道客单价是多少”;
- **多表关联:** 订单表里只有“用户工号”, 从花名册里把“姓名、部门、手机号”跨表拼过来。

Pandas 实战: 清洗过滤、分组与跨表合并

第一章: 核心代码清单

【1. 缺失清洗与条件过滤】

```
import pandas as pd

orders = pd.DataFrame({
    "订单号": ["A01", "A02", "A03", "A04"],
    "金额": [150.0, -5.0, 300.0, None],
    "状态": ["已支付", "已退款", "已支付", "已支付"]
})

# 空白自动补 0 并过滤有效单
orders["金额"] = orders["金额"].fillna(0.0)
clean = orders[(orders["状态"] == "已支付") & (orders
    ["金额"] > 0)]
print(clean)
```

【2. 分组汇总与多表 Merge】

```
# 部门分组求总和
dept_sum = df.groupby("所属部门")["消费金额"].sum().
    reset_index()

# 类似 VLOOKUP 跨表补充姓名
user_info = pd.DataFrame({
    "用户ID": ["U101", "U102", "U103"],
    "姓名": ["张三", "李四", "王五"]
})

merged = pd.merge(df, user_info, on="用户ID", how="
    left")
print(merged[["用户ID", "姓名", "消费金额"]])
```

随堂实战：纽约出租车数据简单处理

第一章: NumPy 与 Pandas 综合小练

【实战背景】

- 拿到一份纽约出租车抽样表 taxi_500.csv (共 500 条)。
- 包含字段：fare_amount (金额)、trip_distance (英里)、passenger_count (乘客数)。

【任务清单】

- ❶ 清洗防脏：缺失乘客数补为 1，并过滤剔除金额 ≤ 0 的异常单。
- ❷ 批量打标：用 NumPy 判断，距离 ≥ 3 英里标为“长途”，否则标为“短途”。
- ❸ 分组统计：用 Pandas 统计长途与短途的订单总量与平均客单价。

【参考练习代码骨架】

```
import pandas as pd, numpy as np

# 读入 500 条样本底表
df = pd.read_csv("taxi_500.csv")

# 1. 缺失补 1，并滤出金额大于 0 的有效订单
df["passenger_count"] = df["passenger_count"].fillna(1)
clean_df = df[df["fare_amount"] > 0].copy()

# 2. NumPy 批量打标：区分长途 / 短途
clean_df["trip_type"] = np._____(
    clean_df["trip_distance"] >= 3.0, "长途", "短途"
)

# 3. Pandas 分组求订单数和平均金额
result = clean_df._____("trip_type").agg(
    order_cnt=("fare_amount", "_____"),
    avg_price=("fare_amount", "_____")
).reset_index()

print(result)
```

MODULE 02

小模型意图识别微调

从中文文本到智能分类

不用死背复杂的微积分与反向传播推导，用通俗直白的“新员工入职培训”闭环，掌握专有业务分类小模型的训练全步骤。

【微调四部曲的核心产出】

- **步骤 1 (准备题库)**: 带标准分类标签的业务话术样本;
- **步骤 2 (翻译数字)**: 分词器将汉字转为数字 ID 张量;
- **步骤 3 (试答案分)**: 模型预测与标准答案对比得出误差 Loss;
- **步骤 4 (纠偏调参)**: 优化器根据差距微调权重，模型越练越准。

通俗理解模型微调：新员工上岗培训

第二章：大白话概念

【用生活常识理解微调】

- 打个生动的比方：

- **预训练底座小模型**（如 MiniRBT）：就像一个刚毕业的大学生，懂汉语语法和基础常识，但完全不懂你们公司的具体业务。
- **微调（Fine-tuning）**：给他发了一本《业务分类话术手册》，让他在入职前做 3 天定向刷题集训。

- **为什么用小模型，而不用几千亿大模型？**

- 几千亿大模型运行极其昂贵，还要配高端 GPU 显卡，回复一次还要几秒钟。
- 小模型专精于分类，普通 CPU 服务器就能跑，单次预测只需 **几毫秒**。

业务场景：客服自动分流

用户提问：“我想查下这期信用卡账单还要还多少钱”

↓ 经过微调训练后的小模型

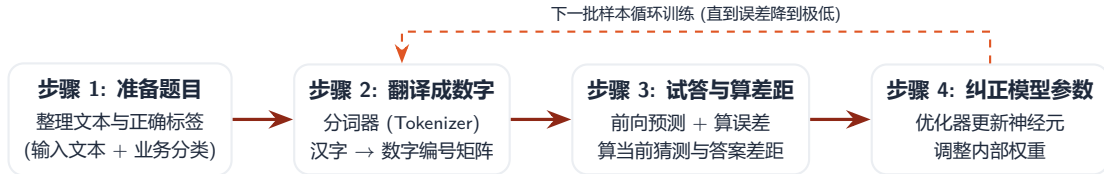
识别出意图：**【还款查询】**（置信度 99%）

↓ 自动联动后续流程

业务流转：系统直接调用还款接口，无需人工客服转接！

微调四步流程：机器是怎么学会分类的？

第二章：模型训练核心闭环



每一步做什么与产生什么结果：

- **步骤 1:** 收集带标签的业务话术 → 产出清洗好的训练数据集 (如: “我要办卡” → “申卡”);
- **步骤 2:** 把中文句子翻译成编号 → 产出模型能够计算的数字矩阵 (`input_ids`);
- **步骤 3:** 模型尝试做选择题并对比标答 → 产出衡量差距的误差值 (`Loss`);
- **步骤 4:** 优化器根据误差调一调内部参数 → 产出对业务分类更准确的模型。

微调四步详细实战与代码呈现

第二章: 完整最小训练闭环

【步骤 1 & 2: 文本转数字】

原始提问: " 查信用卡"

↓ Tokenizer 查字典

编号矩阵: [101, 334, 456, 789, 102]

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("hfl/
minirbt-h256")
encoded = tokenizer("我想查询信用卡账单", padding="
max_length", max_length=16, return_tensors="
pt")
print(encoded["input_ids"])
```

【步骤 3 & 4: 试答与纠错参数】

```
from transformers import
    AutoModelForSequenceClassification
from torch.optim import AdamW

model = AutoModelForSequenceClassification.
    from_pretrained("hfl/minirbt-h256", num_labels=4)
optimizer = AdamW(model.parameters(), lr=2e-5)

# 单步纠偏更新
model.train()
optimizer.zero_grad()
outputs = model(**encoded, labels=target_label)
loss = outputs.loss # 步骤3: 误差差距
loss.backward() # 步骤4: 纠偏
optimizer.step() # 更新模型参数
print(f"误差 Loss: {loss.item():.4f}")
```

MODULE 03

ONNX 极简模型部署

剥离笨重依赖，毫秒级交付

生产服务器无需安装动辄数 GB 的 PyTorch 开发框架，仅需轻量推理引擎，即可实现高吞吐、低延迟上线。

【生产部署的核心动作】

- **模型冻结导出：**通过一行命令将模型转为标准单文件 `model.onnx`;
- **极简生产环境：**容器镜像摆脱深度学习巨型依赖，仅需 `onnxruntime`;
- **多语言极速直调：**CPU 毫秒级返回意图预测，无缝对接 Java/Go/Python 业务中台。

为什么上线部署不要直接用 PyTorch?

第三章: 怎么部署小模型

【开发环境 vs 生产部署的区别】

• 为什么不直接把训练代码搬上线?

- **太臃肿**: PyTorch 安装包动辄 2~3 GB, 占用巨大磁盘与内存。
- **冗余代码多**: 训练需要自动求导和优化器, 但上线只需要“做选择题”, 不需要这些功能。

• 我们希望上线怎么跑?

- **体积小**: 依赖包越小越好。
- **速度快**: 毫秒级给出答案。
- **格式统一**: 导出一个文件, 随处可用。

- **这就是 ONNX 的作用**: 把模型导成一个极小的通用单文件, 线上只装超轻量的 onnxruntime 就能跑!

两种部署方式对比:

直接带 PyTorch 部署 (笨重)

- 服务器需要装 torch (约 2.5GB)
- 启动慢、占用内存大, 难以在微服务中普及。

转为 ONNX 独立部署 (推荐轻量)

- 服务器只需装 onnxruntime (仅几十 MB)
- 模型文件只有一个 model.onnx (约 20MB)
- CPU 毫秒级极速推算, 支持 Java/C++/Python 等各语言调用!

部署解耦架构：从训练环境到生产线上

第三章: ONNX 极简部署全景图



部署核心收益：开发时自由训练，上线时只需将 `model.onnx` 扔到服务器上，无缝对接任何生产系统！

ONNX 怎么用？两步完成上线推理

第三章：生产部署极简两步法

【步骤 1：导出一份独立的 model.onnx】

```
import torch

dummy_input = encoded["input_ids"]
# 一行代码将训练好的模型打包导出
torch.onnx.export(
    model.eval(), dummy_input, "model.onnx",
    input_names=["input_ids"],
    output_names=["output"]
)
```

导出后，服务器只需拷贝这一份文件，开发机上的其他代码和库全都不需要上传。

【步骤 2：纯净生产推理 (无需 torch)】

```
import onnxruntime as ort, numpy as np

# 1. 极轻量运行时加载模型文件
session = ort.InferenceSession("model.onnx")

# 2. 传入数字矩阵并前向预测
inputs = {"input_ids": encoded["input_ids"].numpy()}
logits = session.run(["output"], inputs)[0]

# 3. 极速得出预测业务类别
labels = ["还款查询", "账单明细", "申办卡片", "优惠活动"]
pred = labels[np.argmax(logits)]
print("识别出的业务分类:", pred)
```

全课实战极简清单：遇到业务问题怎么办？

全景速查备忘录

1. 拿到原始业务表格时

- 用 **Pandas** 的 `read_csv` / `read_excel` 读入；
- 用 `fillna()` 快速填补空白值；
- 用 `groupby()` 统计各部门或渠道汇总报表；
- 用 `merge()` 跨表补全用户信息。

2. 需要大批量算账与过滤时

- 用 **NumPy** 替代原生低效 `for` 循环；
- 批量执行全列公式与条件筛选。

3. 遇到客服提问或业务话术理解时

- 选用轻量级底座（如 MiniRBT）；
- 用自己的业务话术数据微调四步训练；
- 产出专精于自己场景的智能小模型。

4. 模型需要上线交付使用时

- 导出为 **ONNX** 单一标准文件；
- 在生产服务器用 `onnxruntime` 运行；
- 摆脱几 GB 的庞大依赖，毫秒级快速响应。

把握核心工具分工，在实践中让业务数据与小模型真正产生价值！

作业：意图识别服务工程化集成

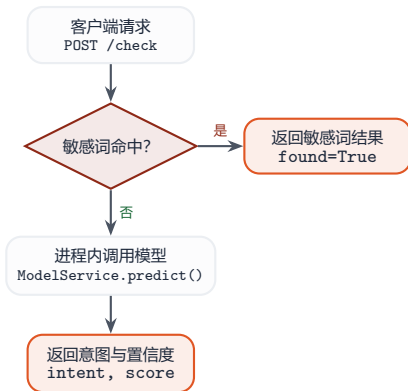
业务流转：单进程常驻

【核心设计思想：极简单进程架构】

- **常驻内存，拒绝重复加载：**在 FastAPI 的 lifespan 启动生命周期中一次性载入 ONNX 模型与分词器，常驻内存供所有请求直接调用。
- **代码层物理隔离：**模型推理不单独开放任何 HTTP 接口，外部网络无法直连模型，消灭跨服务网络序列化开销与安全漏洞。
- **敏感词联合拦截：**请求经由唯一业务入口 POST /check，执行“敏感词优先检查 → 未命中才调用模型”的串联决策链路。

业务判定规则：若命中敏感词，直接返回敏感词结果 (found=True, 意图字段为空)；只有合规安全的文本，才进入模型预测返回意图与置信度！

【业务入口 /check 内部决策流】



作业：意图识别服务工程化集成

增量修改、本地测试与镜像交付

【代码改造清单 (增量开发不破坏 Lab 2)】

- **新增** `model_service.py`:
 - 实现 `load()`: 读取 manifest, 本地加载 Tokenizer 与 ONNX 运行时会话;
 - 实现 `predict()`: 完成 NFKC 文本规整、编码、Session 运行及 Softmax 概率解码。
- **修改** `src/utils/config.py`: 新增模型路径环境变量 `MINIRBT_BUNDLE_DIR` 注入。
- **修改** `src/check/api.py`: 在 `lifespan` 挂载单例, 在 `/check` 串联意图识别。
- **保持原样**: 严禁改动 Lab 2 的 Redis 与敏感词文件, 杜绝代码冲突。

【环境交付与验收标准】

- **依赖瘦身**: `requirements.txt` 仅补充 `onnxruntime`、`transformers` 与 `numpy`, 容器内严禁打包笨重的 `torch`。
- **单元测试通过 (本地验收)**: 运行 `pytest` 验证文本归一化、Softmax 概率和与意图推断分支:

```
export MINIRBT_BUNDLE_DIR=/path/to/bundle
cd src && python -m pytest ../test/test_lab3.py
-v
```

- **容器封装 (Docker)**: 在 `Dockerfile` 中通过 `COPY` 植入模型 Bundle 并配准环境变量。

避坑提醒: 严禁在每次请求里调用 `load()`; 模型初始化只在服务启动时触发一次, 单句推理延迟应在数毫秒级。



中国银联
China UnionPay

Python 实战大课堂 · 2026

Python 实战大课堂·2026